

Positional Encoding

সম্পূর্ণ টেকনিক্যাল গাইড

Transformer Architecture-এ Positional Encoding কী, কেন দরকার, এবং Sine-Cosine ফাংশন দিয়ে এটি ঠিক কীভাবে কাজ করে — একেবারে ছোট বিষয় থেকে গভীরে।

Transformer

Self-Attention

Sine/Cosine

NLP

Deep Learning

Attention is All You Need

📌 বিষয়সূচি

1. Self-Attention-এর দুটি বড় সুবিধা ও একটি বড় সমস্যা
2. Positional Encoding কেন দরকার?
3. সহজ সমাধান: সরল Counting — এবং তার ৩টি সমস্যা
4. Sine ও Cosine কী? — একদম গোড়া থেকে
5. Sine Function দিয়ে Positional Encoding
6. Sine + Cosine পেয়ার — Vector Representation
7. আসল সূত্র: "Attention is All You Need" পেপার
8. হাতে-কলমে হিসাব: "river bank" উদাহরণ
9. Heatmap গ্রাফ বোঝা
10. Binary Encoding-এর সাথে সাদৃশ্য
11. Relative Position — মাইন্ড-রোয়িং প্রপারটি
12. সম্পূর্ণ প্রবাহ: End-to-End Flow
13. সারসংক্ষেপ ও মূল বিষয়গুলো

SECTION 01



Self-Attention-এর দুটি বড় সুবিধা ও একটি বড় সমস্যা

Positional Encoding বোঝার আগে আমাদের জানতে হবে Self-Attention Module কী করতে পারে এবং কোথায় এটি ব্যর্থ হয়। এটি না বুঝলে Positional Encoding-এর প্রয়োজনীয়তাই বোঝা যাবে না।

✓ স্মৃতি ১: Contextual Embedding

ধরো সেন্টেন্স: "river bank" এবং "money bank"।

Self-Attention বুঝতে পারে যে প্রথমটায় "bank" মানে নদীর তীর, দ্বিতীয়টায় মানে ব্যাংক। একই শব্দের অর্থ context অনুযায়ী পরিবর্তন হয় — এটাই Dynamic/Contextual Embedding।

✓ স্মৃতি ২: Parallel Processing

RNN-এ শব্দ একটা একটা করে পাঠাতে হয়। ১০,০০০ শব্দ মানে ১০,০০০ ধাপ।

Self-Attention-এ সব শব্দ একসাথে পাঠানো যায়। ১০,০০০ শব্দও একসাথে প্রসেস হয় — অনেক দ্রুত।

✗ বড় সমস্যা: শব্দের ক্রম (Order) বোঝা না!

যেহেতু সব শব্দ একসাথে পাঠানো হয়, Self-Attention বুঝতে পারে না কোন শব্দটা আগে আর কোনটা পরে।

উদাহরণ:

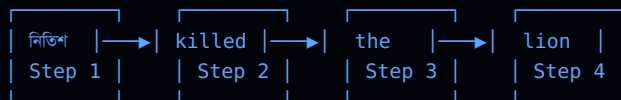
🐘 "নিতিশ killed the lion" — নিতিশ বাঁচে

🦁 "The lion killed নিতিশ" — নিতিশ মরে

Self-Attention-এর কাছে এই দুটো সেন্টেন্স একই! কারণ উভয়েই একই তিনটি শব্দ — শুধু ক্রম আলাদা।

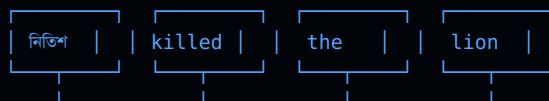
এটি NLP-তে বিপজ্জনক। অর্থ পুরোপুরি বদলে যেতে পারে শব্দের ক্রম পরিবর্তনে।

RNN (Sequential – ধীরে ধীরে):



✓ ক্রম বোঝে ✗ ধীর (Sequential)

Self-Attention (Parallel – একসাথে):



✓ দ্রুত (Parallel) ✗ ক্রম জানে না!



Positional Encoding কেন দরকার?

সমস্যাটা স্পষ্ট — Self-Attention Parallel Processing-এর জন্য শব্দের ক্রম হারিয়ে ফেলে। তাহলে সমাধান কী?

🔑 **মূল ধারণা:** প্রতিটি শব্দকে তার Embedding-এর সাথে একটি অতিরিক্ত তথ্য দিতে হবে — "তুমি কততম স্থানে আছো।" এই তথ্য এনকোড করাকেই বলে **Positional Encoding**।

📌 বাস্তব জীবনের উদাহরণ

ধরো একটা রেস্তোরাঁয় ১০টি টেবিল আছে। ওয়েটার যদি প্রতিটি অর্ডার ট্রে-তে রাখে কিন্তু কোন ট্রে কোন টেবিলের সেটা না লেখে — পুরো রান্নাঘর গুণগোল হয়ে যাবে।

Positional Encoding = প্রতিটি ট্রে-তে টেবিল নম্বরের লেখ। এতে রান্নাঘর (Self-Attention) জানতে পারে কোন খাবার কোথায় পাঠাতে হবে।

Positional Encoding হলো সেই লজিক যা Self-Attention-কে জানায় — কোন শব্দ সেটেলসে কততম স্থানে আছে।

SECTION 03



সহজ সমাধান: সরল Counting — এক তার ৩টি সমস্যা

প্রথমে সবচেয়ে সহজ ধারণা থেকে শুরু করা যাক। "First Principles Approach" — যেন আমরা প্রথমবার এটা আবিষ্কার করছি।

🔑 সহজ সমাধান: সংখ্যা যোগ করে

সেটেল: "নিতিশ killed the lion"

প্রতিটি শব্দের Embedding Vector-এর শেষে তার position নম্বর যোগ করে দাও:

শব্দ	Embedding (512 মাত্রা)	Position যোগ	নতুন Size
নিতিশ	[e1, e2, e3, ..., e512]	+ [1] →	513 মাত্রা
killed	[e1, e2, e3, ..., e512]	+ [2] →	513 মাত্রা
the	[e1, e2, e3, ..., e512]	+ [3] →	513 মাত্রা
lion	[e1, e2, e3, ..., e512]	+ [4] →	513 মাত্রা

এটা দেখতে সহজ মনে হচ্ছে। কিন্তু এতে ৩টি মারাত্মক সমস্যা আছে:

✗ সমস্যা ১: **Unbounded (সীমহীন সংখ্যা)**

ছোট সেটঙ্গে শেষ নম্বর হবে ৪। কিন্তু একটা ৫০০ পৃষ্ঠার বইয়ের PDF-এ লক্ষাধিক শব্দ থাকলে শেষ নম্বর হবে ১,০০,০০০ বা তার বেশি।

Neural Network-এর সমস্যা: Neural Network Backpropagation-এর উপর কাজ করে।

Backpropagation বড় সংখ্যা পছন্দ করে না। -1 থেকে 1-এর মধ্যে সংখ্যা থাকলে ভালো। বড় সংখ্যায় Vanishing Gradient বা Exploding Gradient সমস্যা হয়।

Normalization করলে ও সমস্যা: প্রতিটি position-কে মোট শব্দ সংখ্যা দিয়ে ভাগ করলে 0-1 পাওয়া যাবে। কিন্তু "thank you" (2 শব্দ)-এর দ্বিতীয় শব্দের ভ্যালু = $2/2 = 1.0$, আর "নিতিশ killed the lion" (4 শব্দ)-এর দ্বিতীয় শব্দের ভ্যালু = $2/4 = 0.5$ । একই position (দ্বিতীয়) কিন্তু আলাদা ভ্যালু — Neural Network কনফিউজ হয়।

✗ **সমস্যা ২: Discrete Values (ভাঙা ভাঙা সংখ্যা)**

1, 2, 3, 4 — এগুলো Discrete সংখ্যা। মানে এক থেকে সরাসরি দুই-এ লাফ দেয়, মাঝখানে কোনো মান নেই।

Neural Network-এর সমস্যা: Neural Network Smooth, Continuous পরিবর্তন পছন্দ করে। Discrete সংখ্যায় Gradient Flow ভালো হয় না, যা Numerical Instability তৈরি করে।

✗ **সমস্যা ৩: Relative Position ধরতে পারে না**

Absolute Position মানে: নিতিশ = 1ম, killed = 2য়, the = 3য়, lion = 4র্থ — প্রতিটির নিজস্ব স্বতন্ত্র নম্বর আছে। এটা পারে।

Relative Position মানে: "the" শব্দটি "নিতিশ" থেকে ঠিক ২ ধাপ পরে আছে — এই দূরত্বের তথ্য। এটা পারে না।

কারণ: Discrete Function থেকে আসা আউটপুট দিয়ে Model-এর পক্ষে $3-1=2$ বুঝতে পারা কঠিন। এর জন্য দরকার Continuous/Periodic Function।

✓ **তাহলে আমাদের কী দরকার?**

- **Bounded:** সংখ্যা -1 থেকে 1-এর মধ্যে থাকতে হবে
- **Continuous:** মসৃণ পরিবর্তন হতে হবে, হঠাৎ লাফ নয়
- **Periodic:** পুনরাবৃত্তিমূলক হতে হবে, যাতে Relative Position বোঝা যায়

এই তিনটি গুণ একসাথে আছে এমন **Mathematical Function** হলো → **Trigonometric Functions (Sine, Cosine)**

SECTION 04



Sine ও Cosine কী? — একদম গোড়া থেকে

এই পুরো Positional Encoding-এর হৃদয় হলো Sine এবং Cosine । এগুলো না বুঝলে বাকি কিছুই বোঝা যাবে না । তাই এটা একদম গোড়া থেকে বুঝি ।

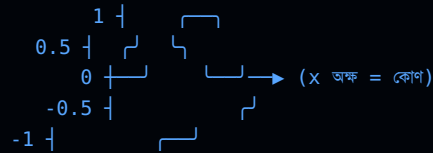
● Sine (sin) কী?

কল্পনা করো একটি চাকা (বৃত্ত) ঘুরছে । সেই চাকার উপর একটি বিন্দু আছে । সেই বিন্দুর উচ্চতা (উপরে-নিচে যাওয়া) যদি আঁকো — সেটাই Sine Curve ।

বৃত্ত (Circle):



Sine Curve:

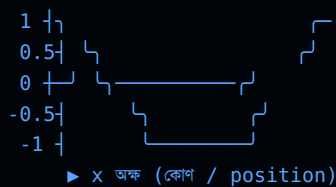


- সর্বোচ্চ মান: **+1** (বিন্দু একদম উপরে)
- সর্বনিম্ন মান: **-1** (বিন্দু একদম নিচে)
- সুতরাং Sine সবসময় **-1** এবং **+1** এর মধ্যে থাকে → **Bounded** ✓
- বিন্দুর উচ্চতা মসৃণভাবে পরিবর্তন হয় → **Continuous** ✓
- একবার পূর্ণ ঘুরলে একই উচ্চতায় ফিরে আসে → **Periodic** ✓

● Cosine (cos) কী?

একই চাকার উপর বিন্দুর **ডানে-বামে** সরে (অনুভূমিক দূরত্ব) আঁকলে সেটা Cosine Curve ।

Cosine Curve:



- Cosine-ও **-1** এবং **+1** এর মধ্যে → **Bounded** ✓
- মসৃণ পরিবর্তন → **Continuous** ✓
- Periodic → **Periodic** ✓
- Sine-এর তুলনায় **90°** আগে শুরুর করে (Phase Shift)

⚠ Sine ও Cosine একসাথে কেন?

একটু পরে বিস্তারিত বোঝাবো, তবে সংক্ষেপে:

যদি শুধু Sine ব্যবহার করো, তাহলে দুটো ভিন্ন position-এর মান একই হতে পারে (কারণ Sine Periodic)। কিন্তু Sine এবং Cosine দুটো একসাথে দিলে — একটা Vector তৈরি হয় যেখানে দুটো মান একসাথে same হওয়ার সম্ভাবনা অনেক কমে যায়।

এছাড়াও, Sine ও Cosine একসাথে থাকলে একটা গাণিতিক সুন্দর প্রপারটি তৈরি হয় যা Relative Position ধরতে পারে — এটা পরে বিস্তারিত দেখবো।

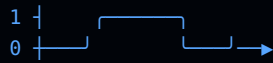
🎵 Frequency (কম্পাঙ্ক) কী?

Sine Curve কত দ্রুত ওঠা-নামা করে সেটাই Frequency।

High Frequency (দ্রুত ওঠানামা):



Low Frequency (ধীর ওঠানামা):



→ High Frequency = তাড়াতাড়ি পরিবর্তন হয়

→ Low Frequency = ধীরে পরিবর্তন হয়

Positional Encoding-এ আমরা বিভিন্ন Frequency-র Sine-Cosine ব্যবহার করব। প্রতিটি জোড়ার Frequency আলাদা — এতে ছোট position এবং বড় position আলাদাভাবে চেনা যায়।

SECTION 05



Sine Function দিয়ে Positional Encoding

এখন আমরা Sine Function দিয়ে Position encode করার চেষ্টা করি।

🔮 পদ্ধতি

সেটেন্স: "নিতিশ killed the lion"

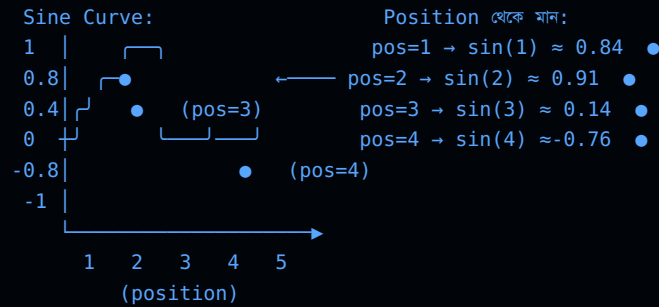
→ নিতিশ = position 1 → $y = \sin(1) \approx 0.84$

→ killed = position 2 → $y = \sin(2) \approx 0.91$

→ the = position 3 → $y = \sin(3) \approx 0.14$

→ lion = position 4 → $y = \sin(4) \approx -0.76$

এই ভ্যালুগুলো প্রতিটি Embedding-এর শেষে যোগ করো।



সরল Counting-এর ৩টি সমস্যা এখন সমাধান হলো:

✓ Bounded

$\sin$ -এর মান সবসময় -1 থেকে $+1$ এর মধ্যে। বড় সেটেলোও সীমার মধ্যে।

✓ Continuous

$\sin$ একটি মসৃণ (smooth) Curve। কোনো হঠাৎ লাফ নেই।

✗ নতুন সমস্যা: একই মান দুবার আসতে পারে!

Sine Periodic। মানে একসময়ের পর একই ভ্যালু ফিরে আসে। যেমন $\sin(1) \approx \sin(1 + 2\pi) \approx \sin(1 + 4\pi) \dots$

সুতরাং position 1 এবং position 7 (বা 35)-এর মান একই হতে পারে! তাহলে Model কীভাবে বুঝবে কোনটা 1ম আর কোনটা 7ম?

এই সমস্যার সমাধান: শুধু Sine নয়, Sine এবং Cosine দুটো একসাথে ব্যবহার করতে হবে।

SECTION 06



Sine + Cosine সেয়ার — Vector Representation

Sine শুধু ব্যবহার করলে collision হয়। তাই এখন Sine এবং Cosine জোড়া ব্যবহার করবো।

১

প্রতিটি শব্দের জন্য দুটো মান নিই

"নিতিশ" (position 1)-এর জন্য: $\sin(1) \approx 0.84$ এবং $\cos(1) \approx 0.54$

এখন এটি Scalar নয়, একটি **Vector**: **[0.84, 0.54]**

২ একই মান আসার সম্ভাবনা কমলো

দুটো position-এ দুটোই মান একসাথে একই হওয়া অনেক বেশি কঠিন। Scalar-এ একটা মিললেই collision, Vector-এ দুটো একসাথে মিলতে হবে।

৩ আরো বেশি পেয়ার যোগ করো

তবুও যদি বড় document-এ collision হওয়ার সম্ভাবনা থাকে, তাহলে আরো Sine-Cosine পেয়ার যোগ করো। Vector আরো বড় হবে।

পেয়ার ১: $\sin(\text{pos}/1)$, $\cos(\text{pos}/1)$ ← High Frequency (দ্রুত ওঠানামা)
পেয়ার ২: $\sin(\text{pos}/2)$, $\cos(\text{pos}/2)$ ← কিছুটা কম Frequency
পেয়ার ৩: $\sin(\text{pos}/4)$, $\cos(\text{pos}/4)$ ← আরো কম Frequency
পেয়ার ৪: $\sin(\text{pos}/8)$, $\cos(\text{pos}/8)$ ← Low Frequency (ধীর ওঠানামা)
...
পেয়ার N: $\sin(\text{pos}/\text{large})$, $\cos(\text{pos}/\text{large})$ ← Very Low Frequency

প্রতিটি পেয়ার থেকে ২টি সংখ্যা → Vector বড় হয়

4 পেয়ার → 8 মাত্রার Vector

64 পেয়ার → 128 মাত্রার Vector

256 পেয়ার → 512 মাত্রার Vector (Original Paper)

গুরুত্বপূর্ণ পর্যবেক্ষণ: Frequency কমেতে থাকে

লক্ষ্য করো — প্রতিটি পরবর্তী পেয়ারের Frequency আগেরটার চেয়ে কম। প্রথম পেয়ার দ্রুত ওঠানামা করে, শেষ পেয়ার ধীরে ধীরে পরিবর্তন হয়।

এটা কেন জরুরি — সেটা Heatmap অংশে বুঝবো।

SECTION 07



আসল সূত্র: "Attention is All You Need" পেপার

এতক্ষণ আমরা ধারণাটা বুঝলাম। এখন দেখি "Attention is All You Need" পেপারে (page 6) ঠিক কী সূত্র ব্যবহার হয়েছে।

$$PE(\text{pos}, 2i) = \sin\left(\text{pos} / 10000^{2i/d_{\text{model}}}\right)$$

$$PE(\text{pos}, 2i+1) = \cos\left(\text{pos} / 10000^{2i/d_{\text{model}}}\right)$$

প্রতিটি চিহ্নের অর্থ:

চিহ্ন	পুরো নাম	অর্থ	উদাহরণ
<code>pos</code>	Position of the Word	শব্দটি সেটের কততম স্থানে — 0 থেকে শুরু	"river" = 0, "bank" = 1
<code>i</code>	Dimension index (pair index)	কোন পেয়ার — 0 থেকে $(d_model/2 - 1)$ পর্যন্ত	6 মাত্রায় $i = 0, 1, 2$
<code>2i</code>	Even dimension	জোড় মাত্রার index (Sine ব্যবহার হয়)	0, 2, 4, ...
<code>2i+1</code>	Odd dimension	বেজোড় মাত্রার index (Cosine ব্যবহার হয়)	1, 3, 5, ...
<code>d_model</code>	Model Dimension	Embedding Vector-এর মাত্রা সংখ্যা	Original Paper: 512
<code>10000</code>	Base constant	Frequency কমানোর জন্য ব্যবহৃত বড় সংখ্যা	পেপারে নির্বাচিত

❓ 10000 কেন?

$10000^{(2i/d_model)}$ এই অংশটি Frequency নিয়ন্ত্রণ করে। $i=0$ হলে এটি 1 হয় (high frequency), i বাড়লে এটি বড় হয় (lower frequency)। 10000 একটি বড় সংখ্যা হওয়ায় শেষ দিকের পেয়ারগুলো খুব ধীরে পরিবর্তন হয় — এতে দীর্ঘ document-এও প্রতিটি position-এর Encoding আলাদা থাকে।

🔧 Vector Dimension সবসময় d_model -এর সমান

পেপারে Embedding Vector 512 মাত্রার। তাই Positional Encoding Vector-ও 512 মাত্রার হবে। তাহলে পেয়ার সংখ্যা $= 512/2 = 256$ টি Sine-Cosine পেয়ার।

+ Addition, ন Concatenation কেন?

Embedding Vector এবং Positional Encoding Vector — এই দুটো যোগ (Add) করা হয়, জোড়া লাগানো (Concatenate) নয়।

কারণ: Concatenate করলে $512+512 = 1024$ মাত্রার Vector হতো। এতে Neural Network-এর Parameters দ্বিগুণ হয়ে যেত — Training সময় দ্বিগুণ! Add করলে Vector size একই থাকে (512), তাই Computational cost বাড়ে না।

```
Embedding Vector (d_model = 6):
river = [e1, e2, e3, e4, e5, e6 ]
```

+

```
PE Vector (d_model = 6):  
river = [PE0, PE1, PE2, PE3, PE4, PE5]
```

=

```
Final Vector (d_model = 6): ← আকার একই!  
river = [e1+PE0, e2+PE1, e3+PE2, e4+PE3, e5+PE4, e6+PE5]
```

▼

Self-Attention Block
(এতে Embedding + Position দুটো তথ্যই আছে)

SECTION 08



হাতে-কলমে হিসাব: "river bank" উদাহরণ

সূত্র বুঝলাম। এখন হাতে-কলমে হিসাব করি। সেটেলস: **"river bank"**, $d_model = 6$ (সহজ করার জন্য)।

সেটআপ:

- "river" = position 0 (pos=0)
- "bank" = position 1 (pos=1)
- $d_model = 6 \rightarrow$ তাই $i = 0, 1, 2$ (মোট 3 পেয়ার, 6 মাত্রা)

হিসাব: "river" (pos = 0)

$i=0 \rightarrow$ Dimension 0 (Sine) * Dimension 1 (Cosine)

ভেতরের ভাগ = $pos / 10000^{(2i/d_model)} = 0 / 10000^{(0/6)} = 0/1 = 0$ $PE(0, 0) = \sin(0) = 0.0 \leftarrow$
Dimension 0 $PE(0, 1) = \cos(0) = 1.0 \leftarrow$ Dimension 1

$i=1 \rightarrow$ Dimension 2 (Sine) * Dimension 3 (Cosine)

ভেতরের ভাগ = $0 / 10000^{(2/6)} = 0 / 10000^{(0.333)} = 0 / 21.54 = 0$ $PE(0, 2) = \sin(0) = 0.0 \leftarrow$
Dimension 2 $PE(0, 3) = \cos(0) = 1.0 \leftarrow$ Dimension 3

$i=2 \rightarrow$ Dimension 4 (Sine) * Dimension 5 (Cosine)

ভেতরের ভাগ = $0 / 10000^{(4/6)} = 0 / 10000^{(0.667)} = 0 / 464.16 = 0$ $PE(0, 4) = \sin(0) = 0.0 \leftarrow$
Dimension 4 $PE(0, 5) = \cos(0) = 1.0 \leftarrow$ Dimension 5

"river"-এর Positional Encoding Vector = [0.0, 1.0, 0.0, 1.0, 0.0, 1.0]

pos=0 হওয়ায় প্রতিটি sin-এ 0 ঢোকে $\rightarrow \sin(0)=0, \cos(0)=1$ । তাই 0,1 বারবার আসে।

হিসাব: "bank" (pos = 1)

i=0

ভেতরের ভালু = $1 / 10000^{(0/6)} = 1 / 1 = 1$ PE(1, 0) = $\sin(1) = 0.8415 \leftarrow \text{Dimension } 0$ PE(1, 1) = $\cos(1) = 0.5403 \leftarrow \text{Dimension } 1$

i=1

ভেতরের ভালু = $1 / 10000^{(2/6)} = 1 / 21.54 = 0.04642$ PE(1, 2) = $\sin(0.04642) = 0.04640 \leftarrow \text{Dimension } 2$ PE(1, 3) = $\cos(0.04642) = 0.99892 \leftarrow \text{Dimension } 3$

i=2

ভেতরের ভালু = $1 / 10000^{(4/6)} = 1 / 464.16 = 0.002155$ PE(1, 4) = $\sin(0.002155) = 0.002155 \leftarrow \text{Dimension } 4$ PE(1, 5) = $\cos(0.002155) = 0.999998 \leftarrow \text{Dimension } 5$

"bank"-এর Positional Encoding Vector $\approx [0.841, 0.540, 0.046, 0.999, 0.002, 1.000]$

চূড়ান্ত ফলাফল:

"river" (pos=0) PE = [0.000, 1.000, 0.000, 1.000, 0.000, 1.000]

"bank" (pos=1) PE = [0.841, 0.540, 0.046, 0.999, 0.002, 1.000]

লক্ষ্য করো:

→ প্রথম জোড়ায় (i=0) পার্থক্য বেশি: 0.841 vs 0.000 (High Frequency)

→ শেষ জোড়ায় (i=2) পার্থক্য খুব কম: 0.002 vs 0.000 (Low Frequency)

এখন প্রতিটির Embedding-এ Add করো:

river_final = river_embedding + [0.0, 1.0, 0.0, 1.0, 0.0, 1.0]

bank_final = bank_embedding + [0.841, 0.540, 0.046, 0.999, 0.002, 1.0]

এই দুটো Self-Attention Block-এ যায় →

Block জানে river 0ম আর bank 1ম স্থানে।

SECTION 09



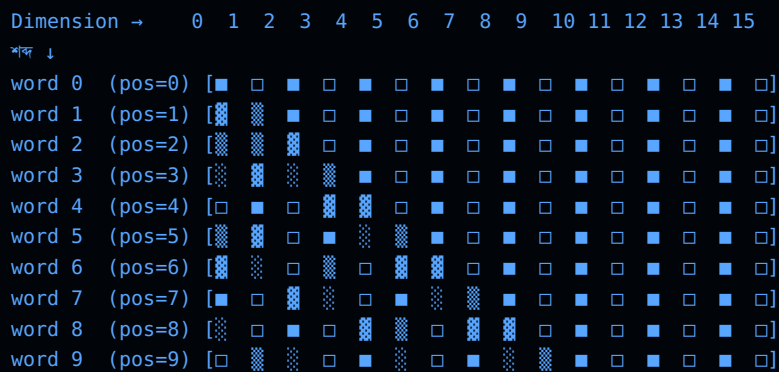
Heatmap গ্রাফ বোঝা

"Attention is All You Need" পেপারে এবং বিভিন্ন রিসোর্সে একটি বিখ্যাত Heatmap দেখা যায়। এটা কী বোঝায়?

Heatmap-এর সেটআপ (উদাহরণ)

- সেটলে 50টি শব্দ
- প্রতিটি শব্দের Embedding 128 মাত্র (d_model=128)
- তাই 50টি Positional Encoding Vector, প্রতিটি 128 মাত্র
- X-axis = 128টি Dimension, Y-axis = 50টি শব্দ
- রঙ = সেই মাত্রার Positional Encoding মান (উজ্জ্বল=+1, গাঢ়=-1)

Heatmap (সরলীকৃত, 10 শব্দ × 16 মাত্রা):



কিংবদন্তী: □=-1 ■=-0.5 ■=0 ■=+0.5 ■=+1

১. গুরুত্বপূর্ণ পর্যবেক্ষণ ১: প্রথম word (pos=0)

pos=0 হলে সব Sine মান = $\sin(0)=0$ (সাদা/গাঢ়), সব Cosine মান = $\cos(0)=1$ (উজ্জ্বল)। তাই প্রথম সারিতে পরিষ্কার 0,1,0,1,... প্যাটার্ন দেখা যায়।

২. গুরুত্বপূর্ণ পর্যবেক্ষণ ২: Lower Dimensions-এ বেশি পরিবর্তন

Heatmap-এর বাম দিকে (Lower Dimensions = High Frequency Sine/Cosine) প্রতিটি শব্দে রঙ বেশি পরিবর্তন হয়। ডান দিকে (Higher Dimensions = Low Frequency) রঙ প্রায় একই থাকে।

কারণ: High Frequency মানে দ্রুত ওঠানামা — পাশাপাশি শব্দেও মান আলাদা। Low Frequency মানে ধীর পরিবর্তন — অনেক শব্দ পার হলে তবে পরিবর্তন হয়।

উপমা: ঘড়ির সেকেন্ডের কাঁটা (High Frequency — দ্রুত বদলায়), মিনিটের কাঁটা (মাঝারি), ঘণ্টার কাঁটা (Low Frequency — ধীরে বদলায়)।

৭. গুরুত্বপূর্ণ পর্যবেক্ষণ ৩: আরো শব্দ হল **Higher Dimensions**-ও সক্রিয় হয়

50 শব্দের সেট্টেলে হয়তো শেষের Dimension খুব কম পরিবর্তন হয়। কিন্তু 500 বা 1000 শব্দের document-এ Higher Dimensions-ও পরিবর্তন দেখাবে — কারণ X-axis (position) আরো বড় হবে।

SECTION 10

12
34

Binary Encoding-এর সাথে আদৃশ্য

এই পুরো Positional Encoding-এর একটি অসাধারণ উপমা আছে। এটি Binary Encoding-এর মতো — কিন্তু Continuous সংখ্যার ভেতনে।

Binary Encoding (Discrete)

0 → 0000 1 → 0001 ← LSB (Last bit) প্রতিবার বদলায় 2 → 0010 3 → 0011 4 → 0100 ← 2nd bit প্রতি 2 বারে বদলায় 5 → 0101 6 → 0110 7 → 0111 8 → 1000 ← MSB খুব কম বদলায়

✗ Discrete — Neural Network পছন্দ করে না

Positional Encoding (Continuous)

pos=0 → [0.000, 1.000, 0.000, 1.000, ...] pos=1 → [0.841, 0.540, 0.046, 0.999, ...] pos=2 → [0.909, -0.416, 0.093, 0.996, ...] pos=3 → [0.141, -0.990, 0.139, 0.990, ...] ... ← Low-dim দ্রুত বদলায় (LSB-এর মতো) ← High-dim ধীরে বদলায় (MSB-এর মতো)

✓ Continuous — Neural Network পছন্দ করে!

🌀 মূল অন্তর্দৃষ্টি:

Binary Encoding-এ LSB (ভানদিকের বিট) প্রতিটি সংখ্যায় বদলায়। MSB (বাঁদিকের বিট) খুব কম বদলায়। ঠিক একইভাবে Positional Encoding-এ প্রথম Dimension (High Frequency Sine) প্রতিটি position-এ দ্রুত বদলায়, শেষ Dimension (Low Frequency Sine) ধীরে বদলায়।

পার্থক্য শুধু একটাই: Binary Discrete, কিন্তু Positional Encoding Continuous।

Binary Encoding থেকে অনুপ্রাণিত Positional Encoding:

Binary:	bit3	bit2	bit1	bit0
	MSB			LSB
0	0	0	0	0

```
1      0      0      0      1  ← LSB বদলায়
2      0      0      1      0  ← LSB+1 বদলায়
...
পরিবর্তনের হার: LSB > bit1 > bit2 > MSB
```

Positional Encoding:

	Dim4-5	Dim2-3	Dim0-1
	LowFreq		HighFreq
pos=0	≈ 1.0	≈ 1.0	$\sin/\cos(0)$
pos=1	≈ 1.0	≈ 1.0	$\sin/\cos(1)$ ← High Freq dim বদলায়
pos=10	≈ 1.0	≈ 0.99	$\sin/\cos(10)$
pos=100	≈ 1.0	≈ 0.9	...

পরিবর্তনের হার: HighFreqDim > MidFreqDim > LowFreqDim

SECTION 11



Relative Position — মাইন্ড-ব্লোয়িং প্রপারটি

এটি সবচেয়ে গভীর এবং চমকপ্রদ অংশ। মনে আছে, সহজ Counting সমাধানের তৃতীয় সমস্যা ছিল Relative Position ধরতে পারে না? এখন দেখবো Sine-Cosine পদ্ধতি কীভাবে এটা সমাধান করে।

🔑 মূল দাবি

এই Positional Encoding পদ্ধতিতে, প্রতিটি নির্দিষ্ট দূরত্ব (Δ)-এর জন্য একটি নির্দিষ্ট **Linear Transformation (Matrix)** আছে। মানে:

যদি আমার কাছে **PE(pos)** ভেক্টর থাকে, তাহলে একটি নিরুদ্ভিষ্ট **Matrix M_{Δ}** দিয়ে গুণ করলে **PE(pos+ Δ)** পাওয়া যাবে।

➤ উদাহরণ: $\Delta=10$ এর Matrix

একটি নির্দিষ্ট Matrix M_{10} আছে এমন যে:

$$M_{10} \times \text{PE}(10) = \text{PE}(20)$$

$$M_{10} \times \text{PE}(3) = \text{PE}(13)$$

$$M_{10} \times \text{PE}(21) = \text{PE}(31)$$

একই Matrix — যেকোনো position থেকে ঠিক 10 এগিয়ে দেয়!

➤ উদাহরণ: $\Delta=5$ এর Matrix

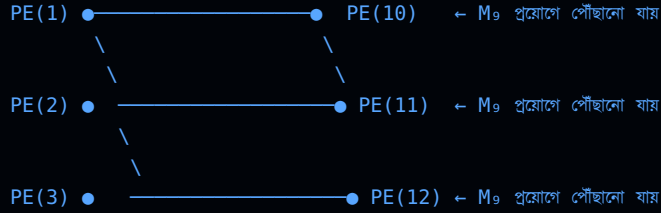
$$M_5 \times \text{PE}(5) = \text{PE}(10)$$

$$M_5 \times \text{PE}(17) = \text{PE}(22)$$

এই Matrix সবসময় 5 এগিয়ে দেয়।

প্রতিটি Δ -এর জন্য একটি Linear Transformation আছে। Neural Network এই Transformation শিখতে পারে। তাই সে বুঝতে পারে দুটো শব্দের মধ্যে Relative দূরত্ব কত।

128-মাত্রার Vector Space-এ (2D-তে কল্পনা করো):



একই M_9 Matrix দিয়ে যেকোনো $PE(k)$ থেকে $PE(k+9)$ -এ পৌঁছানো যায়।
এই ধর্মই প্রমাণ করে Relative Position এনকোড করা আছে।

🔗 Mathematical Insight: কেন Sine+Cosine দুটোই লাগে?

এই Linear Transformation Matrix-টি দেখতে এরকম (এটি Rotation Matrix):

$M_{\Delta} = \begin{bmatrix} \cos(\Delta/\omega) & -\sin(\Delta/\omega) \\ \sin(\Delta/\omega) & \cos(\Delta/\omega) \end{bmatrix}$ যেখানে $\omega = 10000^{2i/d_{\text{model}}}$ কারণ:
 $PE(\text{pos}+\Delta) = M_{\Delta} \times PE(\text{pos})$ $\begin{bmatrix} \sin(\text{pos}+\Delta)/\omega \end{bmatrix} = \begin{bmatrix} \cos(\Delta/\omega) & -\sin(\Delta/\omega) \end{bmatrix} \times \begin{bmatrix} \sin(\text{pos}/\omega) \end{bmatrix}$
 $\begin{bmatrix} \cos(\text{pos}+\Delta)/\omega \end{bmatrix} \begin{bmatrix} \sin(\Delta/\omega) & \cos(\Delta/\omega) \end{bmatrix} \begin{bmatrix} \cos(\text{pos}/\omega) \end{bmatrix}$ এটা কাজ করে কারণ: $\sin(A+B) = \sin A \cos B + \cos A \sin B$
 $\cos(A+B) = \cos A \cos B - \sin A \sin B$

সারকথা: Sine-Cosine পেয়ার ব্যবহার করলে এই Rotation Matrix তৈরি করা সম্ভব। শুধু Sine থাকলে এই সুন্দর গাণিতিক সম্পর্ক তৈরি হতো না। এই কারণেই Sine ও Cosine দুটোকে পেয়ার করে রাখা হয়।

🧠 চমকপ্রদ সত্য: এই পুরো ব্যবস্থা মূলত Binary Encoding-কে Continuous সংখ্যার জগতে নিয়ে আসার মতো — এবং সেই সাথে Relative Position Capture করার ক্ষমতা দেয়। Researchers-রা বলছেন এটি "মাইন্ড-ব্লোয়িং"।

SECTION 12



সম্পূর্ণ প্রবাহ: End-to-End Flow

এখন পুরো প্রক্রিয়াটি একসাথে দেখি।

INPUT SENTENCE: "নিতিশ killed the lion"

ধাপ ১: Tokenization

নিতিশ	killed	the	lion
pos = 0	pos = 1	pos = 2	pos = 3

ধাপ ২: Word Embedding ($d_{\text{model}} = 512$)

নিতিশ $\rightarrow [e_1, \dots, e_{512}]$	$\leftarrow$ Embedding Lookup Table থেকে
killed $\rightarrow [e_1, \dots, e_{512}]$	
the $\rightarrow [e_1, \dots, e_{512}]$	
lion $\rightarrow [e_1, \dots, e_{512}]$	

ধাপ ৩: Positional Encoding ($d_{\text{model}} = 512$)

প্রতিটি pos-এর জন্য PE Vector (512 মাত্রা) তৈরি:

PE(0):	$[\sin(0/\omega_0), \cos(0/\omega_0),$	
	$\sin(0/\omega_1), \cos(0/\omega_1),$	
	$\dots$	256 পেরার]
PE(1):	$[\sin(1/\omega_0), \cos(1/\omega_0),$	
	$\sin(1/\omega_1), \cos(1/\omega_1),$	
	$\dots$	256 পেরার]
$\dots$		

যেখানে $\omega_i = 10000^{(2i/512)}$

ধাপ ৪: Addition (Embedding + PE)

নিতিশ_final = নিতিশ_embedding + PE(0) [512 মাত্রা]

killed_final = killed_embedding + PE(1) [512 মাত্রা]

the_final = the_embedding + PE(2) [512 মাত্রা]

lion_final = lion_embedding + PE(3) [512 মাত্রা]

ধাপ ৫: Self-Attention Block

[নিতিশ_final, killed_final,	
the_final, lion_final]	
$\downarrow$	
Self-Attention	
(এতে শব্দের অর্থ + অবস্থান দুটোই আছে)	

✓ এই পদ্ধতির সুবিধাগুলো একত্রে

গুণ	কীভাবে পাওয়া যায়	কেন দরকার
Bounded (-1 to 1)	Sine/Cosine সবসময় -1 থেকে +1	Neural Network Stable থাকে
Continuous	Sine/Cosine smooth curve	Gradient Flow ভালো হয়
Unique per position	বহু পেরার $\rightarrow$ Vector collision কঠিন	প্রতিটি position আলাদা

Relative Position	Linear Transformation (Rotation Matrix)	দুই শব্দের দূরত্ব বোঝা যায়
Same size as Embedding	d_model মাত্রার PE Vector	Addition করা সম্ভব

SECTION 13



সারসংক্ষেপ ও মূল বিষয় গুলো

📌 মূল পয়েন্ট — একনজরে

- **সমস্যা:** Self-Attention Parallel Processing করে, তাই শব্দের ক্রম বোঝে না।
- **সমাধান:** প্রতিটি শব্দের Embedding-এ Positional Encoding Vector যোগ করে।
- কেন **Sine-Cosine:** Bounded, Continuous, Periodic — তিনটি গুণই আছে।
- **Vector আকার:** PE Vector-এর মাত্রা = Embedding-এর মাত্রা (d_model)।
- **সেয়ার পদ্ধতি:** প্রতি দুটো মাত্রার জন্য একটি Sine-Cosine পেয়ার, ক্রমবর্ধমান Frequency-তে।
- **সূত্র:** $PE(pos, 2i) = \sin(pos/10000^{(2i/d_model)})$, $PE(pos, 2i+1) = \cos(\text{একই})$ ।
- **Addition না Concatenation:** Add করলে Vector size বাড়ে না, Parameter কম লাগে।
- **Relative Position:** Rotation Matrix-এর মাধ্যমে $PE(pos)$ থেকে $PE(pos+\Delta)$ পাওয়া যায়।
- **উপমা:** Binary Encoding-এর Continuous সংস্করণ।

🔗 Interview-এ কী জিজ্ঞেস হতে পারে?

- Positional Encoding কেন দরকার? → Self-Attention ক্রম বোঝে না।
- Concatenation করা হয় না কেন? → Parameter ও Training Time দ্বিগুণ হতো।
- কেন Sine-Cosine পেয়ার? → Relative Position Capture করার Rotation Matrix-এর জন্য।
- Frequency কমতে থাকে কেন? → বিভিন্ন "scale"-এ position চেনার জন্য (Binary Encoding-এর মতো)।

📖 আরো গভীরে জানতে

Tim Dettmers-এর ব্লগ — "Linear Relationships in the Transformer's Positional Encoding" — পড়ো। সেখানে Rotation Matrix-এর গণিত বিস্তারিত আছে। ChatGPT বা অন্য AI দিয়ে কঠিন অংশ সহজ করে বোঝাও।

হাতে-কলমে অনুশীলন: "Hi my name is X" — এই সেন্টেন্সের প্রতিটি শব্দের PE Vector কাগজে হিসাব করো। একবার করলেই সব কনফিউশন দূর হবে।

📖 Positional Encoding Technical Guide — বাংলায় Deep Learning

Based on: "Attention is All You Need" — Vaswani et al., 2017